RRT or Rapidly-exploring Random Tree (wheeled robots, RRT, Prim's algorithm & MST, path and motion planning for solving mazes)

Robotic Systems I

Konstantinos Asimakopoulos *k_asimakopoulos@ac.upatras.gr*
Lampros Printzios *printzios_lampros@ac.upatras.gr*

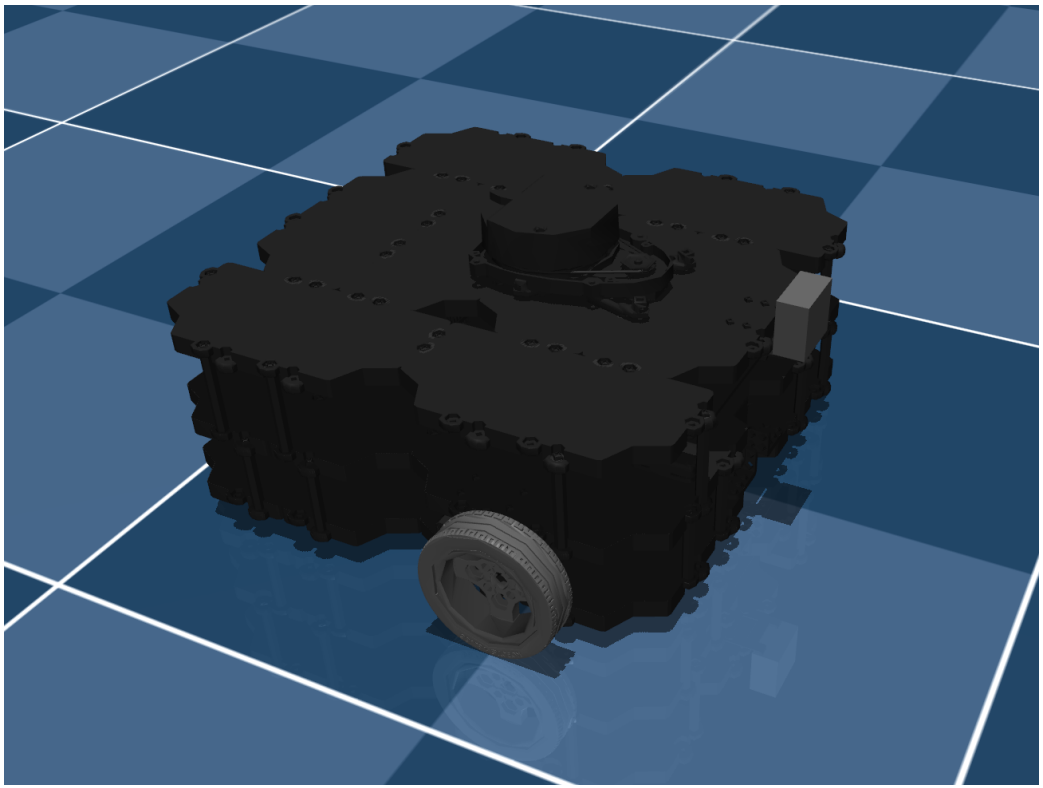
Department of Electrical and Computer Engineering
University of Patras

March 2, 2026

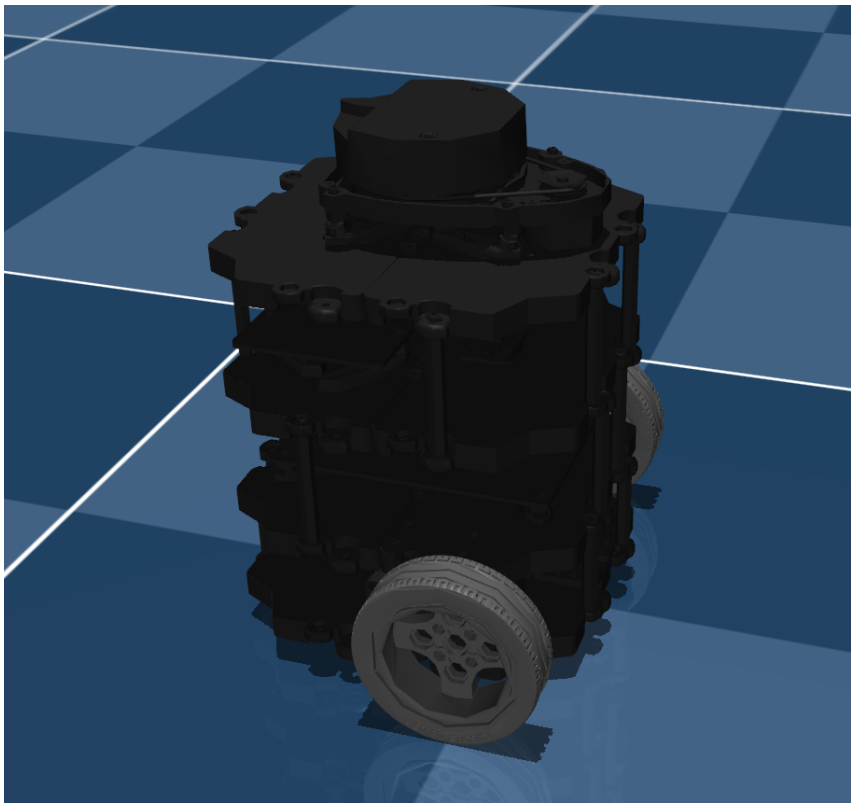
Table of Contents

Differential Drive Robot (DDR)

Typical DDRs are TurtleBot3 Waffle Pi (used in this lab) and TurtleBot3 Burger, that are showed below.



TurtleBot3 Waffle Pi



TurtleBot3 Burger

Differential Drive Robot (DDR) - How it Works?

A **Differential Drive Robot** (DDR) is a type of mobile robot:

1. It moves on the 2D plane, and its state consists of its 2D position (x, y) and its yaw angle orientation θ:

$$\mathbf{x} = \begin{bmatrix} x & y & \theta \end{bmatrix}^\top \in \mathbb{R}^3$$

2. It has **two independently actuated wheels** and usually another **one free turning supporting wheel**. Its motion is generated by the angular speeds ω_L and ω_R of its left and right wheel respectively.
3. Important parameters affecting its motion are the *fixed wheel distance* (separation) **L** and *fixed wheel radius* **r**.
4. It is a **nonholonomic system**, obeying to an important motion constraint: **it can not move laterally** (sideways), assuming no slipping between the wheels and the ground. This is mathematically described by the Pfaffian nonholonomic constraint (on velocity):

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0 \Rightarrow \begin{bmatrix} \sin\theta & -\cos\theta & 0 \end{bmatrix} \dot{\mathbf{x}} = 0$$

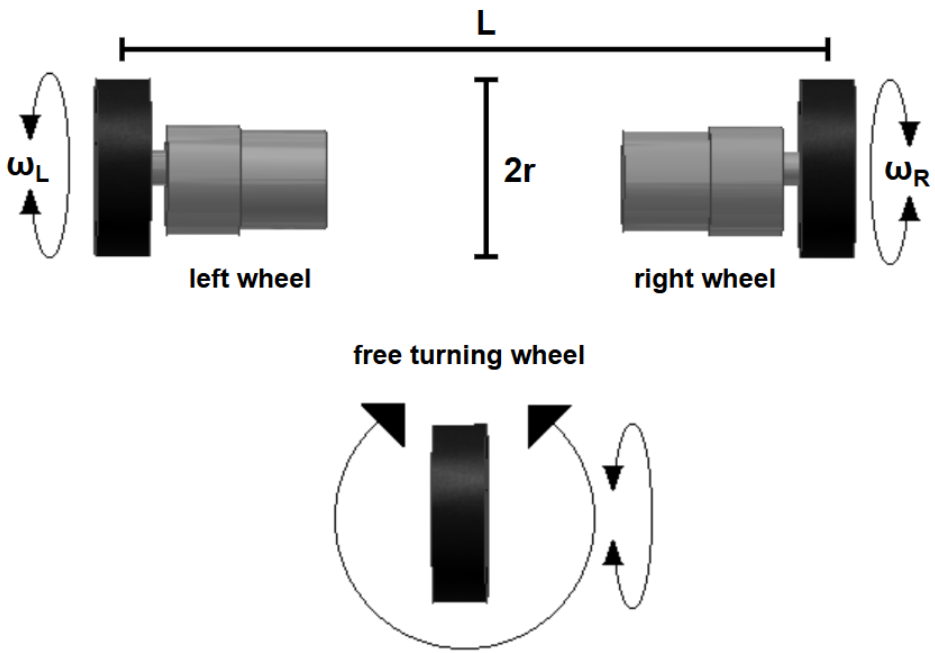
Differential Drive Robot (DDR) - Kinematics of DDR

For linear and angular velocities v, ω respectively of the DDR system, and wheel angular speeds ω_L, ω_R, we can derive the following state equations:

$$\dot{\mathbf{x}} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} \frac{r\cos\theta}{2} & \frac{r\cos\theta}{2} \\ \frac{r\sin\theta}{2} & \frac{r\sin\theta}{2} \\ -\frac{r}{L} & \frac{r}{L} \end{bmatrix} \cdot \begin{bmatrix} \omega_L \\ \omega_R \end{bmatrix}$$

We can achieve desired velocities v, ω, actuating the wheels with the following angular speeds ω_L, ω_R:

$$\begin{aligned} v &= \frac{r}{2}(\omega_R + \omega_L) \\ \omega &= \frac{r}{L}(\omega_R - \omega_L) \end{aligned} \implies \begin{aligned} \omega_R &= \frac{1}{r}v + \frac{L}{2r}\omega \\ \omega_L &= \frac{1}{r}v - \frac{L}{2r}\omega \end{aligned}$$



Differential Drive System Diagram

Differential Drive Robot (DDR) - Other Wheeled Robots

Holonomic robots

- Omnidirectional (mecanum) wheels
- Can move in any direction instantly (no velocity constraint)

Car-like robots

- Ackermann steering
- Turning radius constraint

The type of mobile robots we use here, **differential drive** ones, are:

- Simple
- Cheap
- Very common in research

MyAGV Elephant Robotics (omnidirectional)

Rapidly-exploring Random Tree (RRT) - What it is & Properties

RRT (Rapidly-exploring Random Tree) is a **probabilistic** algorithm designed for efficient **path planning** in *high-dimensional*, *complex*, and *continuous spaces*, commonly used in robotics for **motion planning**. It works by building a **tree** from a start point, expanding toward **random samples**, and **checking for collisions** in the process.

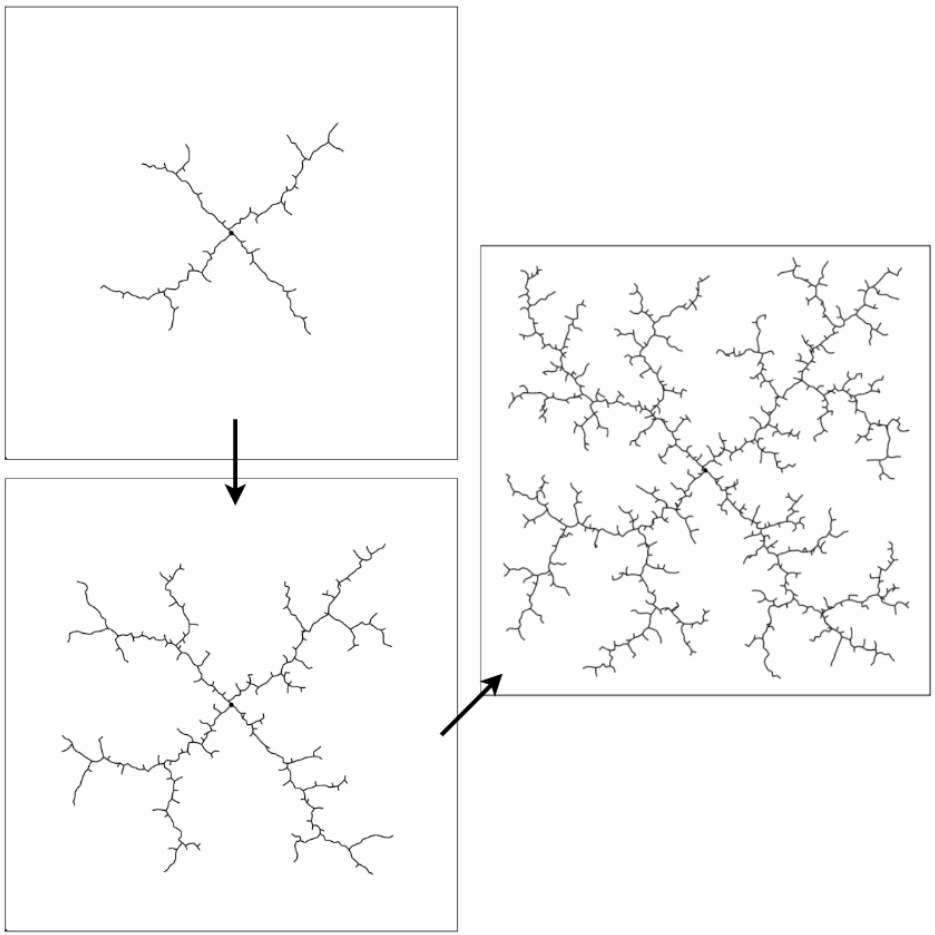
1. Purpose:
   - Find a **collision-free path** in continuous, complex geometries.
   - Work in the continuous state space, **without explicit environment decomposition** (e.g., grid discretization, cell decomposition).
2. Key Properties:
   - **Probabilistically complete** (finds a solution if one exists). It is guaranteed to converge to a **feasible solution** (given enough time), which is generally **not path-optimal, nor control-optimal**. For optimality (at least asymptotic) there is RRT*.
   - **Naturally explores** large unexplored regions.
   - **Easy to implement** in a large variety of systems.

Rapidly-exploring Random Tree (RRT) - Algorithm Description

Assume state space X, free space $X_{free} \subseteq X$, and obstacle space $X_{obs} = X \setminus X_{free}$. RRT builds a tree T incrementally, initializing the search from the node $\mathbf{x}_{start} \in X_{free}$, that is added in T, and then iteratively:

1. Sample random state $\mathbf{x}_{sample} \in X$
2. Find nearest node $\mathbf{x}_{near} \in X_{free}$ in T, to $\mathbf{x}_{sample}$, according to a distance metric
3. Steer from $\mathbf{x}_{near}$ toward sample $\mathbf{x}_{sample}$ (up to a fixed step size) and get a potential new node $\mathbf{x}_{new} \in X$
4. Check for collisions and add the new node in T only if $\mathbf{x}_{new} \notin X_{obs}$

Stop when goal $\mathbf{x}_{goal} \in X_{free}$ is reached (under some error thresholds), add it in T, and retrieve a feasible path from the parent nodes using backtracking.



RRT expansion example

Prim's Algorithm - What it is & Properties

Prim's algorithm is a **greedy** algorithm for constructing a **Minimum Spanning Tree (MST)** of a **weighted, undirected graph**. It starts from an arbitrary start node and incrementally builds a **tree/subgraph** by repeatedly adding the **minimum weight edge** that connects a new vertex to the growing tree/subgraph.

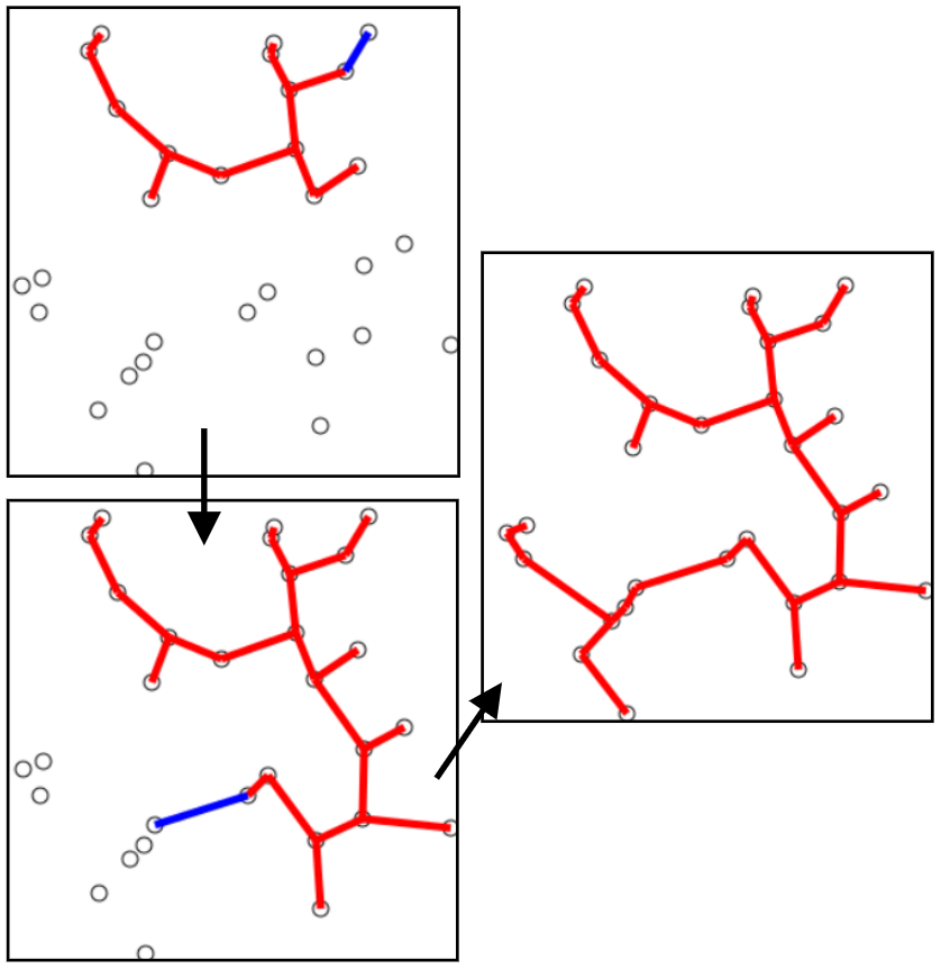
1. Purpose:
   - Construct a **Minimum Spanning Tree (MST)**, by connecting all vertices with the **minimum possible total edge weight**.
2. Key Properties:
   - **Greedy algorithm** (similar in spirit to Kruskal's algorithm).
   - Produces a **fully connected**, **acyclic**, and **undirected** graph (the MST, which is a subgraph of the original graph).
   - Resulting structure contains exactly $|E| = |V| - 1$ edges, where $|V|$ is the number of vertices.

Prim's Algorithm - Algorithm Description

Steps of Prim's algorithm:

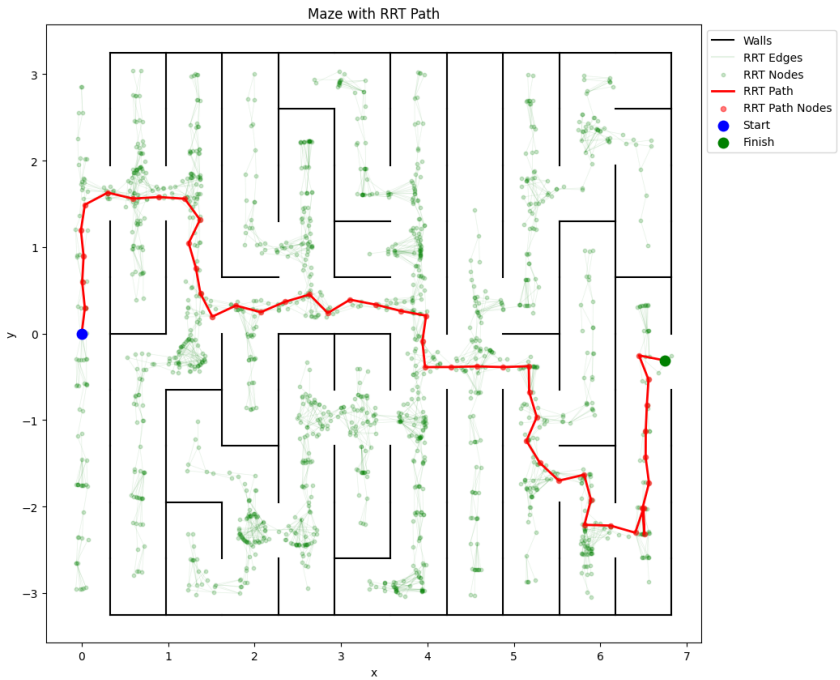
1. Start from an arbitrary vertex
2. Mark it as part of the MST
3. Add all edges from this vertex to a candidate edge list
4. While candidate list is not empty:
   - Select edge with minimum weight (if more than one choose randomly)
   - Remove the selected edge from the candidate list
   - If the edge connects to a vertex not in MST, add that vertex to MST and add its outgoing edges to the candidate list

Continue until all vertices are included in the Minimum Spanning Tree (MST).

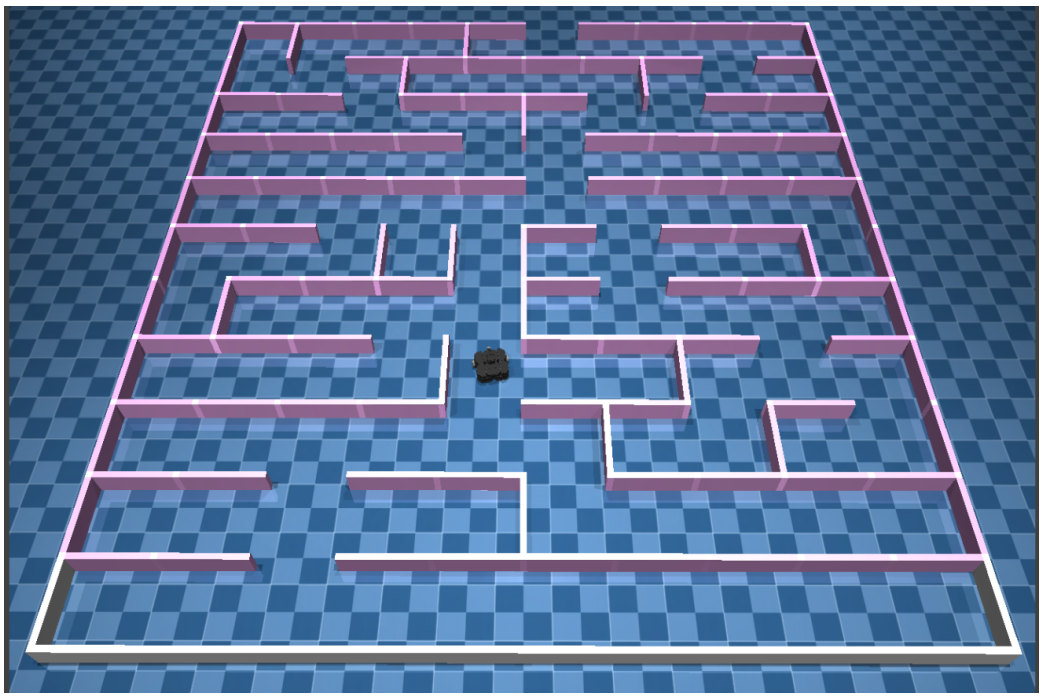


Prim's algorithm MST example

Maze Exercise in MuJoCo



Solving Maze using RRT



Maze and Bot View in MuJoCo

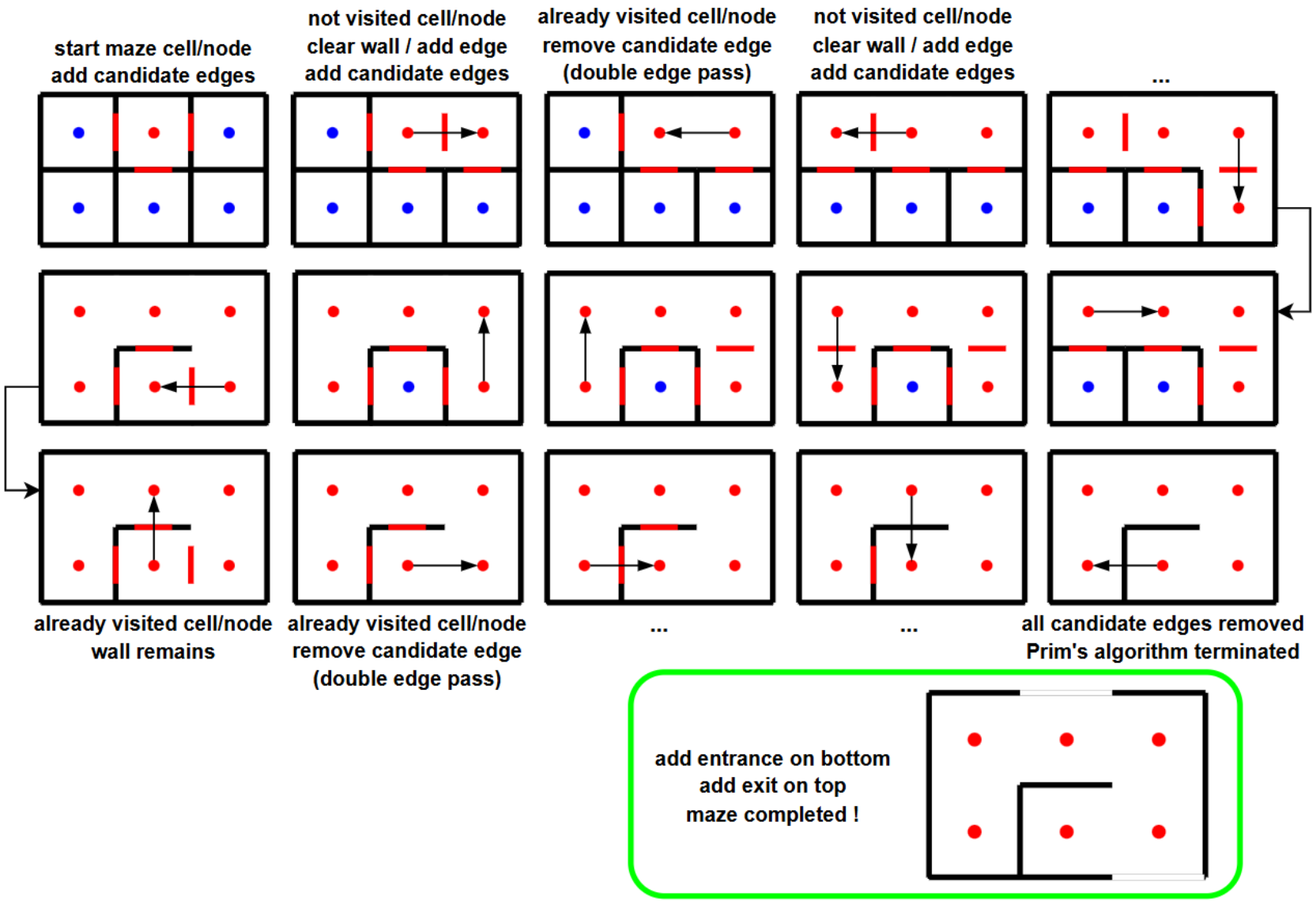
Maze Exercise in MuJoCo - Constructing Random Mazes

We use a **randomized version of Prim's algorithm** for maze generation.

1. Graph Interpretation:
   - *Cell* $\leftrightarrow$ *Node*
   - *Absence of Wall* $\leftrightarrow$ *Edge*
   - *Removing a wall* $\leftrightarrow$ *Adding an edge* to the spanning tree
2. Randomized Prim Variant:
   - We assume all edges have **equal cost**.
   - **No minimum-weight edge comparison** is required.
   - The next edge is chosen applying a **weighted random selection**. We assign directional probabilities/biases P_N (North), P_S (South), P_E (East), P_W (West), for which $P_N + P_S + P_E + P_W = 1$. So, if for example $P_N + P_S > P_E + P_W$, then there is a higher vertical direction probability, which results to longer vertical corridors in the maze.

Prim's algorithm generates a **perfect maze**, meaning that it has exactly one **unique path between any two cells**, with **no loops or inaccessible areas**.

Maze Exercise in MuJoCo - Constructing Random Mazes



Construction steps of a 2×3 maze. Blue and red cells represent the unvisited and visited nodes respectively. Red walls represent the candidate edges left.

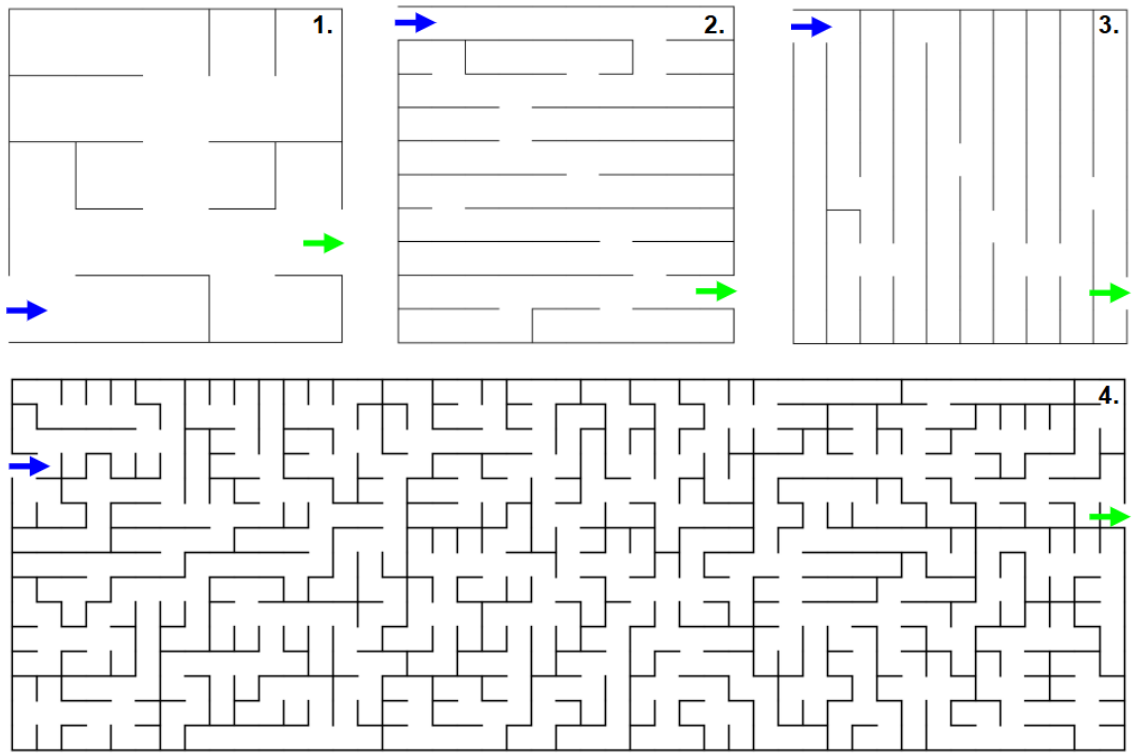
Maze Exercise in MuJoCo - Constructing Random Mazes

Mazes are constructed using a randomized version of Prim's algorithm. Some parameters to experiment with are:

- number of **rows** & **columns**
- **directional probabilities** for Prim's algorithm
- **wall thickness**
- **RNG seed**
- *cell size* (wall length)
- *position* and *orientation* of maze on 2D plane

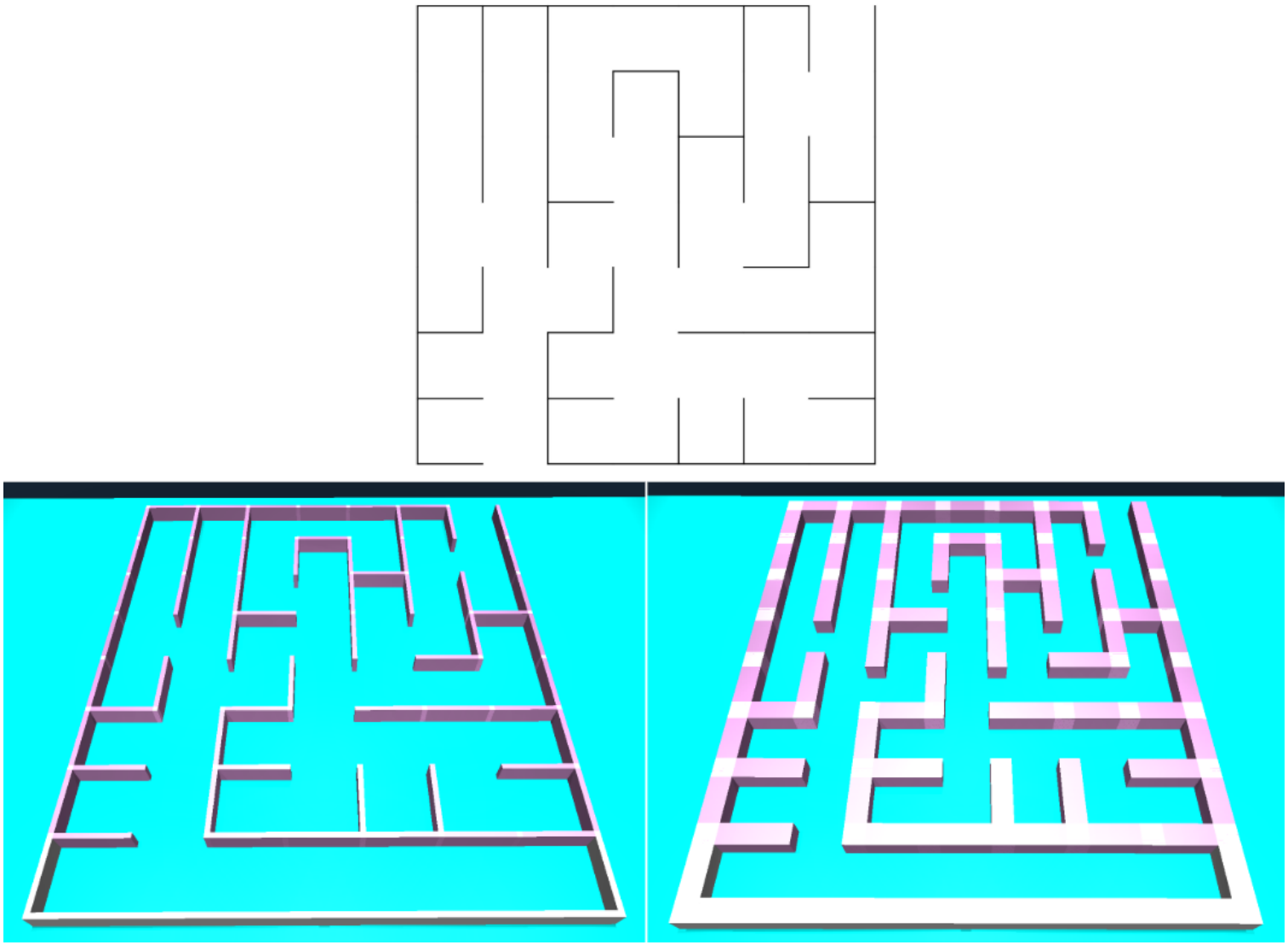
Mazes are exported to:

- **Real Dimensions** XML
- **Inflated-Walls** XML for RRT



Random Mazes examples, with a single entrance and a single exit

Maze Exercise in MuJoCo - Constructing Random Mazes



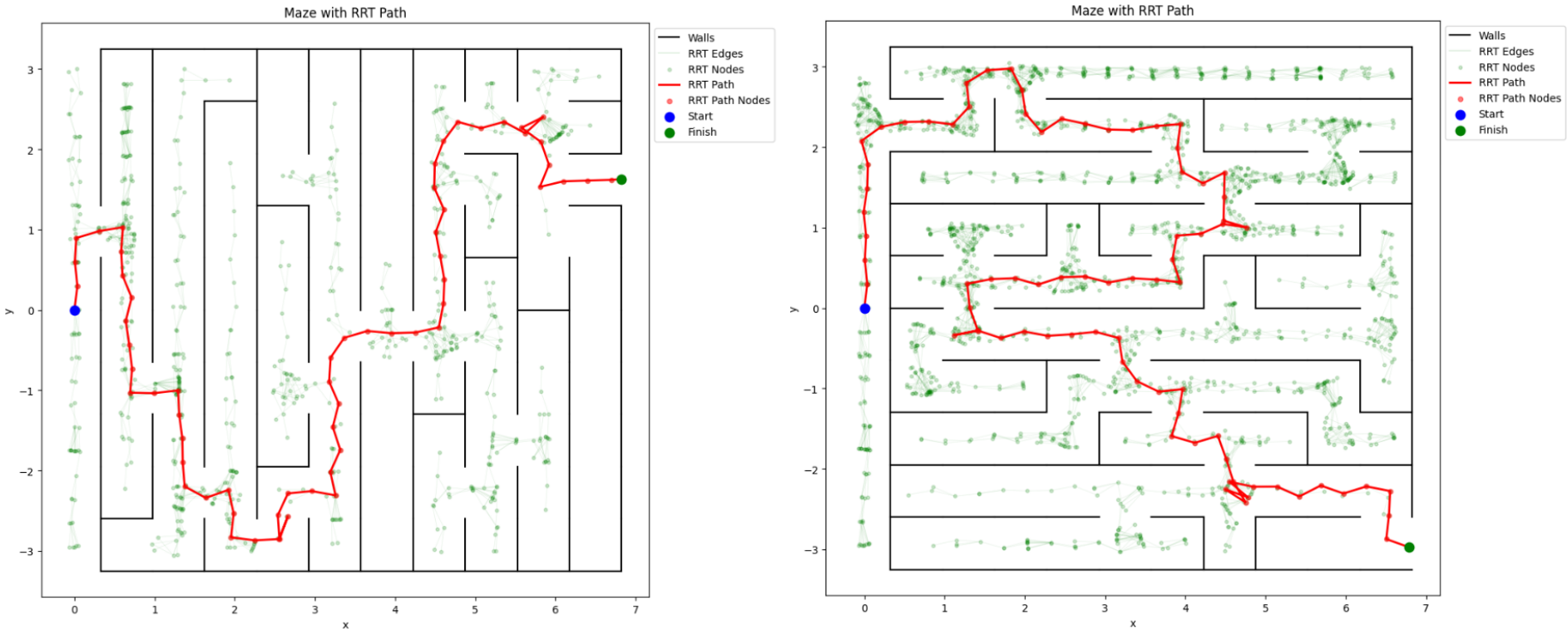
Comparison between real dimensions maze and inflated maze in MuJoCo view.

Maze Exercise in MuJoCo - RRT Path & Motion Planning

1. **Assumptions:**
   - ***Known* maze structure** and **robot state** (via MuJoCo).
   - LiDAR (or any other sensor) not used for state estimation.
2. **Planning in SE(2):**
   - State space: $\mathbf{x} = \begin{bmatrix} x & y & \theta \end{bmatrix}^\top \in \mathbb{R}^3$.
   - **Weighted distance** metric: $d^2 = dx^2 + dy^2 + w_\theta (d\theta)^2$.
   - **Goal sampling bias** and **search area sampling asymmetry** applied.
3. **Collision Checking (Physics-Based):**
   - Use **inflated walls**.
   - **Discretize candidate edge**, set robot pose iteratively for every interpolated point, and run the system forward. If any *contact* is detected between the mobile robot and the walls, *reject the edge*.
4. **Execution Phase:**
   - Load real maze and *follow RRT path* using a **cascaded P-controller**.
   - **Goal condition** (RRT termination condition):

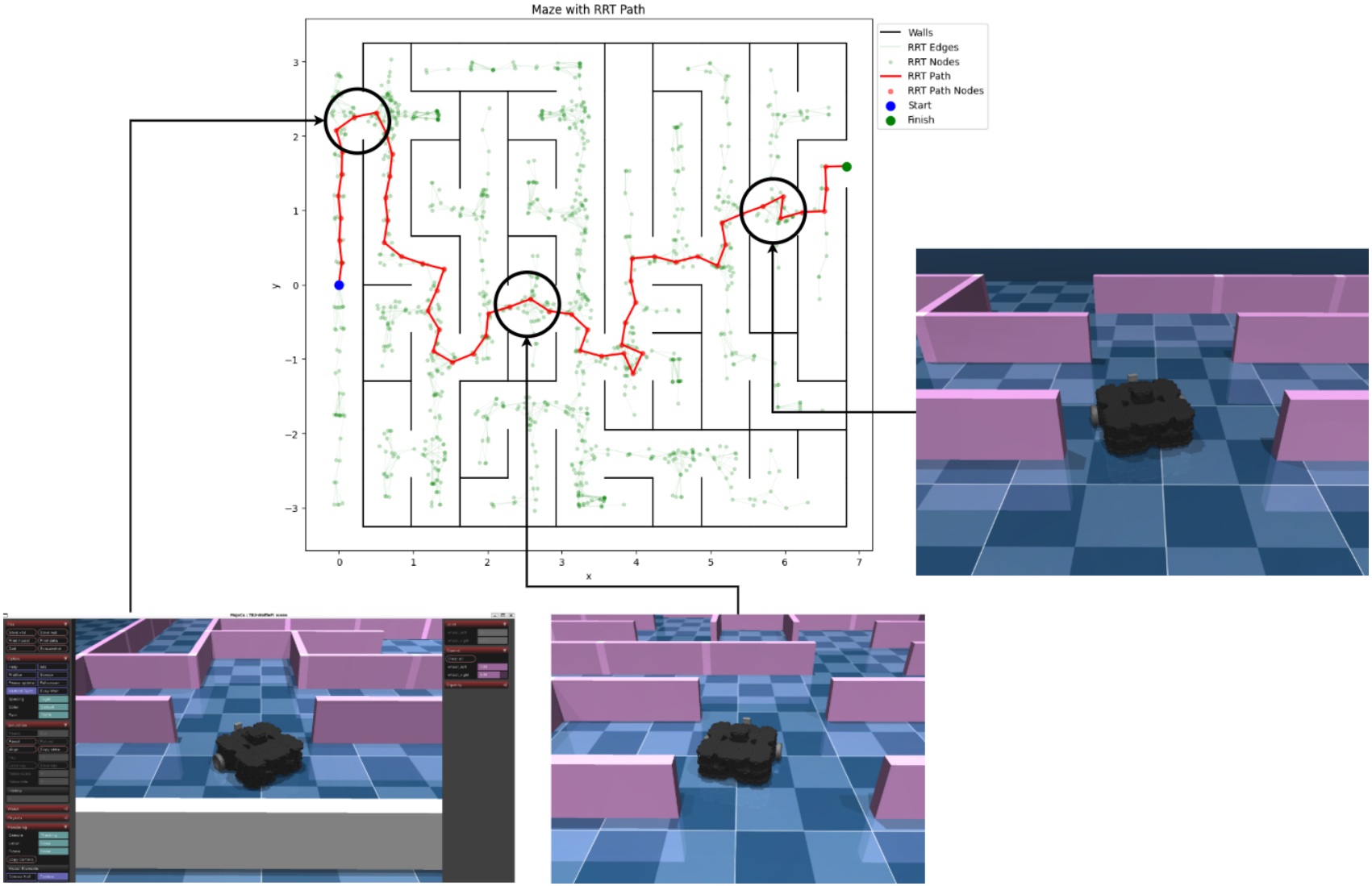
$$\|p - p_{goal}\| < \epsilon_{xy}, \quad |\theta - \theta_{goal}| < \epsilon_\theta$$

Maze Exercise in MuJoCo - RRT Path & Motion Planning



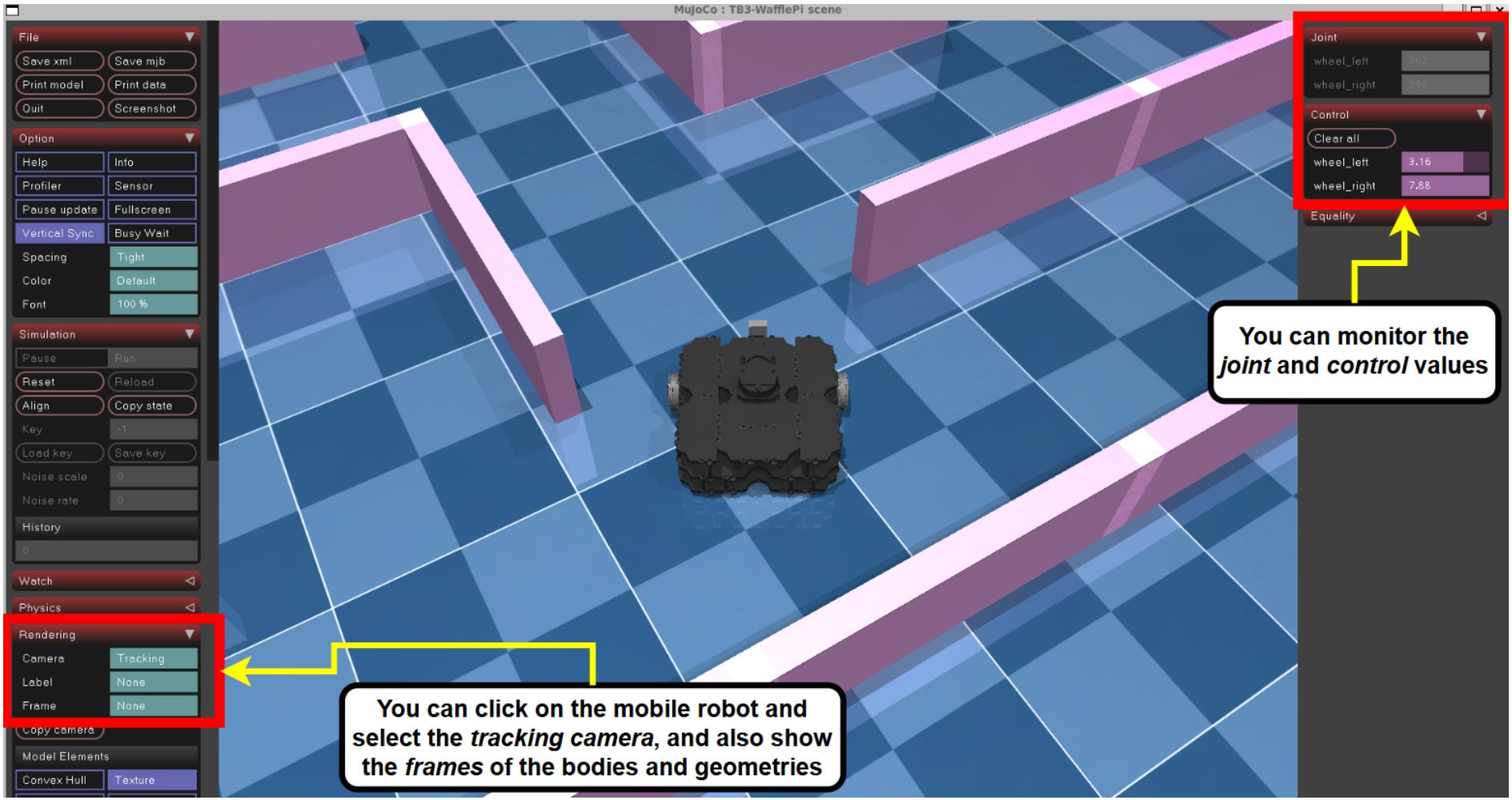
Solving mazes using RRTs. First found feasible paths are shown, along with all the RRT nodes and edges.

Maze Exercise in MuJoCo - Full Example



Full example shown, with the found feasible RRT path from start to finish, and MuJoCo view frames from key points inside the maze

Maze Exercise in MuJoCo - Handling MuJoCo Environment



Showing some MuJoCo utilities

References

- **Differential Drive Robot basic info**
- **RRT original paper "Rapidly-Exploring Random Trees: A New Tool for Path Planning"**
- **Other RRT sources: source1 & source2**
- **Prim's algorithm source1 & source2**
- **MuJoCo documentation**
- **A collection of high-quality models for the MuJoCo physics engine, curated by Google DeepMind (MuJoCo Menagerie)**
- **Modern Robotics: Mechanics, Planning, and Control; Kevin M. Lynch and Frank C. Park**